

Análisis de clientes con Spark

Caso de negocio: desempeño por departamento, categorías, cliente estrella y evolución mensual · Big Data — Entregable 2 · **Integrantes:** Pedro Sierra y Tomás Lopera

T1 · Desempeño por departamento

Ventas totales, ticket promedio y número de ventas por departamento (JOIN transacciones + ciudades, agregación).

Departamento	Ventas totales	Ticket promedio	N° ventas
Cundinamarca	975,402.23	137.25	7,107
Antioquia	825,275.56	138.80	5,946
Valle	419,686.48	140.13	2,995
Atlantico	231,001.21	138.41	1,669
Santander	141,531.31	150.09	943
Bolivar	112,403.37	146.17	769
Risaralda	80,148.87	140.37	571

La suma de las 7 filas es \$2,785,449.03, idéntica al total general: el JOIN no pierde ni duplica información.

- **¿Qué departamento vende más?** Cundinamarca (Bogotá) con \$975,402, seguido de Antioquia (Medellín) con \$825,276. Entre los dos concentran el **65%** de las ventas del año.
- **¿Varía mucho el ticket promedio entre regiones?** No: va de \$137.25 a \$150.09 (~9.4% de dispersión). El comportamiento de compra por transacción es homogéneo en todo el país; la diferencia en ingresos la explica el **volumen** de ventas, no el gasto por transacción.

T2 · Top 3 categorías por ciudad

Ranking de categorías dentro de cada ciudad con ROW_NUMBER() sobre ventana PARTITION BY ciudad.

Ciudad	#1	#2	#3
Bogota	electronica (525,325)	hogar (134,266)	ropa (115,589)
Medellin	electronica (447,358)	hogar (116,633)	ropa (101,595)
Cali	electronica (223,732)	hogar (60,515)	ropa (48,702)
Barranquilla	electronica (123,166)	ropa (33,464)	hogar (32,475)
Bucaramanga	electronica (84,621)	hogar (16,809)	ropa (16,010)
Cartagena	electronica (67,209)	hogar (13,175)	ropa (10,991)
Pereira	electronica (39,293)	hogar (11,379)	ropa (10,691)

- **¿La categoría líder es la misma en todas las ciudades?** Sí: electrónica lidera en las 7 ciudades, con un margen de 3x-4x sobre la segunda categoría.
- **¿Hay diferencias regionales?** Sí: en la mayoría hogar es #2 y ropa #3, pero en **Barranquilla** ropa supera a hogar — el mix de inventario en la costa Caribe debería diferenciarse del resto del país.

T3 · Cliente estrella por ciudad

Cliente de mayor gasto acumulado por ciudad (ventana + JOIN), comparado contra el gasto promedio de su ciudad.

Ciudad	Cliente	Gasto total	Prom. ciudad	Veces prom.
Bogota	106	4,422.55	1,219.25	3.63x
Medellin	298	4,040.62	1,031.59	3.92x
Cali	215	3,002.68	538.06	5.58x

Ciudad	Cliente	Gasto total	Prom. ciudad	Veces prom.
Bucaramanga	346	2,435.94	261.13	9.33x
Cartagena	617	2,034.95	242.25	8.40x
Barranquilla	72	1,978.48	327.66	6.04x
Pereira	479	1,575.17	197.90	7.96x

- **¿Cuánto gastó el mejor cliente frente al promedio de su ciudad?** En ciudades grandes (Bogotá, Medellín) gasta ~3.6-3.9x el promedio local; en ciudades chicas (Bucaramanga, Pereira, Cartagena) llega a 7.9x-9.3x, porque el gasto se reparte entre menos clientes únicos.
- **Implicación:** el programa de fidelización debería usar un umbral *relativo* al promedio de cada ciudad, no un monto fijo nacional.

T4 · Evolución mensual con acumulado

Ventas por mes y acumulado progresivo (SUM(...) OVER (ORDER BY mes)).

Mes	Ventas del mes	Acumulado
2025-01	253,745.19	253,745.19
2025-02	244,188.98	497,934.17
2025-03	234,270.44	732,204.61
2025-04	227,221.35	959,425.96
2025-05	216,728.68	1,176,154.64 (>1M)
2025-06	220,943.90	1,397,098.54
2025-07	228,274.73	1,625,373.27
2025-08	232,584.26	1,857,957.53
2025-09	241,525.38	2,099,482.91
2025-10	222,964.67	2,322,447.58
2025-11	233,672.03	2,556,119.61
2025-12	229,329.42	2,785,449.03

- **¿Hay meses más fuertes o débiles?** Caída suave de enero (\$253,745) a mayo (\$216,729, -15%), recuperación gradual hasta septiembre, leve desaceleración al cierre. Diferencia máx-mín ~17%: sin estacionalidad marcada.
- **¿Qué mes superó el millón acumulado?** Mayo de 2025, con acumulado de \$1,176,154.64.

Parte de optimización (query elegida: T3, cliente estrella)

1. cache() — reutilización de cómputo

T3 reutiliza el DataFrame agregado `gasto_cliente_ciudad` dos veces (ranking por ventana y promedio por ciudad). Al ejecutar `.cache()` en el compute **Databricks Serverless** asignado, Spark devolvió:

```
[NOT_SUPPORTED_WITH_SERVERLESS] PERSIST TABLE is not supported on serverless compute. SQLSTATE: 0A0000
```

Esto se debe a que Serverless administra su propio caché interno de forma automática y no expone control manual sobre persistencia (bloquea el comando de bajo nivel detrás de `cache()`/`persist()`). Se documentó el comportamiento capturando la excepción en tiempo de ejecución (evidencia de investigación del compute) en vez de forzar un fallo del notebook.

Conceptualmente, el beneficio esperado de `cache()` es evitar que Spark recalculé el `join + groupBy` completos en cada acción sobre el mismo DataFrame: sin cache, la 2ª acción vuelve a leer las tablas y recomputa desde cero; con cache (en un clúster clásico), la 2ª acción leería el resultado ya materializado en memoria, con una mejora de tiempo que crece con el número de reusos.

2. explain() — optimización de Catalyst

Sobre una consulta con dos filtros (`monto > 100` antes del join, `departamento == 'Antioquia'` después del join) y un `groupBy`, el plan físico de `explain(True)` muestra ambos filtros aplicados **inmediatamente después del escaneo de cada fuente** (como `PushedFilters` en el `FileScan`), en vez de esperar a después del join tal como está escrito el código. Esto es **predicate pushdown**: Catalyst reordena y empuja los filtros hacia abajo en el plan para reducir el volumen de datos *antes* de la operación más costosa (el join), en vez de filtrar después.

3. Broadcast join — dónde y por qué

Se usó (automáticamente, por debajo del umbral `spark.sql.autoBroadcastJoinThreshold`, y explícitamente con el hint `broadcast(ciu)`) en **todos los JOIN de T1, T2 y T3**, entre transacciones (20,000 filas) y ciudades (7 filas). Al ser ciudades minúscula, Spark copia la tabla completa a cada executor (`BroadcastHashJoin`) en vez de repartir (`shuffle`) ambas tablas por la red (`SortMergeJoin`), evitando el costo de mover la tabla grande. Es el caso de libro de texto para broadcast join: tabla de dimensión pequeña y estática contra una tabla de hechos grande.

Conclusiones generales

- El negocio está concentrado geográficamente (Bogotá + Medellín = 65% de ventas), pero el ticket promedio es homogéneo (~9% de dispersión): la diferencia regional es de volumen, no de gasto por transacción.
- Electrónica lidera sin excepción en las 7 ciudades; el inventario secundario (hogar/ropa) sí debe ajustarse por región (caso Barranquilla).
- El criterio de "cliente estrella" debe ser relativo al promedio de cada ciudad, no un monto absoluto nacional.
- Las ventas mensuales son estables sin estacionalidad marcada; el acumulado cruza el millón en mayo de 2025.
- Spark (DataFrames + Spark SQL, joins, funciones de ventana, acumulados) resolvió los 4 análisis con una fracción del código que hubiera tomado en MapReduce, apoyado en optimizaciones automáticas de Catalyst (predicate pushdown, broadcast join).